

CS 64 HANDOUT #6

Introduction to Digital Logic Design

Prof. Ziad Matni

*Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture! ☺*

In this reading, we will discuss and clarify:

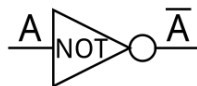
- Basic Logic Symbols for diagraming logic designs
- Truth Tables
- Simplifying and Optimizing Boolean expressions using
 - Truth tables
 - Boolean algebra
 - Simple K-Maps
- Combinatorial Logic example: the Multiplexer (Mux)

1. Basic Logic Symbols and Truth Tables

You already know the basic logic operators: NOT, AND, OR, XOR.

In digital logic design, we have to be able to recognize and draw the symbols of these, and other, logic operations. We often refer to the hardware that handles basic logic as “**gates**”.

A. NOT gate

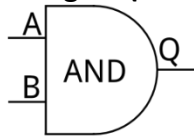


NOTE1: Inputs are shown on the LEFT side and outputs on the RIGHT.

NOTE2: “Bubbles”, like that little circle on the right side of the above symbol, are an indication of binary negation.

The “**bar**” symbol on top of a Boolean variable (e.g. $\bar{A}$) indicates a binary negation to the value of that variable. So, if $A = 0$, then $\bar{A} = 1$ and if $A = 1$, then $\bar{A} = 0$. This is summarized in a table that shows all possible inputs with all possible outputs – it’s called a “**Truth Table**”.

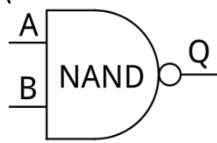
A	$\bar{A}$
0	1
1	0

B. AND gate (and it's opposite, NAND)

This time, we have 2 inputs (A and B). The truth table for **AND** has to show all the possible combinations these inputs (there are 4 such combos, since $2^2 = 4$)

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

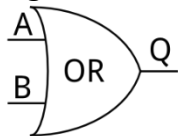
By putting a negation at the output of an **AND** gate, you get its opposite: **NAND** (short for **NOT-AND**). The symbol and truth table for **NAND** is shown here:



A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

Note that we write "**A NAND B**" as $\overline{A \cdot B}$.

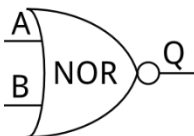
By **DeMorgan's Law**, this is equivalent to: $\overline{A \cdot B} = \overline{A} + \overline{B}$ (i.e., **NOT A OR NOT B**)

C. OR gate

The truth table for **OR**, like the previous ones, has the same number of entries: **4**. That's because there are 2 inputs to a simple OR gate.

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Like **NAND**, the opposite logic of **OR** is **NOT-OR**, or **NOR**:



A	B	$\overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Note that we write "**A NOR B**" as $\overline{A + B}$.

By **DeMorgan's Law**, this is equivalent to: $\overline{A + B} = \overline{A} \cdot \overline{B}$ (i.e., **NOT A AND NOT B**)

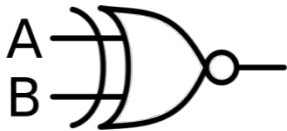
D. XOR gate

The truth table for **XOR** (exclusive-OR) is:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Note that the logic of **XOR** to be true (i.e., for its output = 1), it necessitates that inputs **A** and **B** be UNEQUAL Booleans, that is, for A=0 and B=1, or for A=1 and B=0.

The opposite logic of **XOR** is **NOT-XOR**, or **XNOR**:



A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

Note that the logic of **XNOR** to be true (i.e., for its output = 1), it necessitates that inputs **A** and **B** be EQUAL Booleans, that is, for A=0 and B=0, or for A=1 and B=1.

2. Truth Tables

Truth tables can be applied to ANY digital design and can be extremely useful because they show the outcomes of **ALL** possible combination of inputs.

The number of entries in a truth table is equal to 2^N , where **N** is the number of inputs.

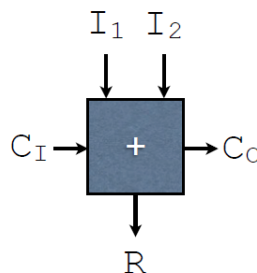
Example 1: a 2-input AND gate has a truth table of 4 entries.

Example 2: a “binary adder” with 3 inputs (**Input₀**, **Input₁**, and **CarryIn**) has a truth table of 8 entries (example coming up...!)

Remember that digital designs can have multiple inputs AND multiple outputs too!

Take, for instance, that 3-input binary adder.

You might envision it like this:



The diagram illustrates this “**box**” (we don’t know what’s inside it) as having 3 inputs (you can tell because of the directions of the arrows) that are called I_1 , I_2 , and C_i . These are 1-bit (i.e. Boolean) values for Input1, Input2, and a Carry-In bit. It also has 2 outputs: R and C_o : the Sum and the Carry-Out bit.

We can *construct a truth table* that has 8 entries (let’s label them 0 thru 7 – note how the combos of the inputs follow the binary values for those labels!) from this abstraction, since we know what the function of it is supposed to be:

	I_1	I_2	C_i	C_o	R
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

As an example of what we did above, look at entry 3:
With the inputs equal to 0, 1, and 1, we add them and get 10 (i.e. binary for 2) – that’s 1 in the Carry-Out (C_o) and 0 in the Sum (R).

The truth table can now help us create **Boolean expressions** for the outputs!

Note that C_o takes on the value 1 (i.e. it is “True”) under exactly **4 conditions**:

- When I_1 , I_2 , and C_i are equal to 0, 1, and 1, respectively (entry 3)
- When I_1 , I_2 , and C_i are equal to 1, 0, and 1, respectively (entry 5)
- When I_1 , I_2 , and C_i are equal to 1, 1, and 0, respectively (entry 6)
- When I_1 , I_2 , and C_i are equal to 1, 1, and 1, respectively (entry 7)

Condition (a) can be written, as a Boolean expression, as: $\bar{I}_1 \cdot I_2 \cdot C_i$ – note that this expression is **True** (i.e. equal to 1) if and only if I_1 , I_2 , and C_i are equal to 0, 1, and 1.

Going through all these other conditions for $C_o = 1$, we see that:

$$C_o = \bar{I}_1 \cdot I_2 \cdot C_i + I_1 \cdot \bar{I}_2 \cdot C_i + I_1 \cdot I_2 \cdot \bar{C}_i + I_1 \cdot I_2 \cdot C_i$$

Going through the same exercise for output R , we see that:

$$R = \bar{I}_1 \cdot \bar{I}_2 \cdot C_i + \bar{I}_1 \cdot I_2 \cdot \bar{C}_i + I_1 \cdot \bar{I}_2 \cdot \bar{C}_i + I_1 \cdot I_2 \cdot C_i$$

Note that the above expressions are written as combinations of logic-**NOT**, **AND**, and **ORs**. Since **OR** is known as a “**logical union**”, it’s been assigned the operation symbol “+” (plus-symbol). This obviously does not mean the same “**addition**”! Likewise, **AND** is known as a “**logical disjunction**” and is assigned the operation symbol “.” (dot-symbol), but it does not mean “**multiplication**”!

However, when we present a Boolean expression written in this form (i.e., like we did for **C₀** and **R**), we have a name for it: “**the sum-of-products form**”.

3. Simplifying Combinatorial Logic Functions

When you combine different logic gates in one Boolean expression, we often call that a “**combinatorial logic function**”.

Now let’s take a look at an interesting example:

$$X = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$$

And let’s write out the truth table for this expression:

A	B	X	
0	0	0	
0	1	1	i.e., this is when $\bar{A} \cdot B$ is True
1	0	1	i.e., this is when $A \cdot \bar{B}$ is True
1	1	1	i.e., this is when $A \cdot B$ is True

Now, does this truth table look familiar to you?? *It should!!* It’s the truth table for **A OR B** !

This is proof enough that actually, **$X = A + B$** .

This simplification has strong implications in digital design.

Simplification – or optimization – processes like this (and there are other, much better, ones that we’ll learn) make a Boolean expression much easier to read and understand. Furthermore, optimization saves computer memory (software) as well as physical space (hardware) – this also has direct implications to saving money on the cost of manufacturing these digital designs. Quite importantly, optimization helps digital designs execute or run *faster*.

In our example above, we found that $X = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$ can be simplified to:

$$X = A + B$$

We went from using 3 **AND**, 2 **OR**, and 2 **NOT** gates (**7** total logic gates) to **just 1 OR gate!!**

4. Simplifying Logic using Boolean Logic Rules (Algebraic Simplification)

In a previous Math or CS course, you may have already learned that **AND** and **OR** logic have both **associative** and **distributive** properties.

$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	<i>associativity of AND</i>
$A + (B + C) = (A + B) + C$	<i>associativity of OR</i>
$A \cdot (B + C) = A \cdot B + A \cdot C$	<i>distributivity of AND over OR</i>
$A + (B \cdot C) = (A + B) \cdot (A + C)$	<i>distributivity of OR over AND</i>

You may have also learned that, if x is a general Boolean value (i.e., it's either **0** or **1**):

$1 \cdot x = x$	$0 \cdot x = 0$
$1 + x = 1$	$0 + x = x$
$x \cdot x = x$	$x \cdot \bar{x} = 0$
$x + x = x$	$x + \bar{x} = 1$

Now, note that $X = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$ can be optimized algebraically as follows:

$$\begin{aligned}
 X &= A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B + \underline{A \cdot B} \quad (\text{NOTE: I just OR'ed that redundant } A \cdot B \text{ at the end}) \\
 &= A \cdot (B + \bar{B}) + B \cdot (\bar{A} + A) \quad (\text{NOTE: According to the distributive law...}) \\
 &= A + B \quad (\text{NOTE: Since } x + \bar{x} = 1)
 \end{aligned}$$

So, we can get the same optimized results using Boolean algebra too.

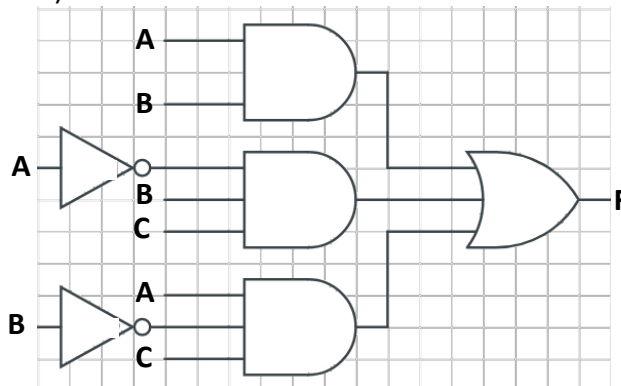
There is one more method that we'll learn for optimization: the **Karnaugh Map** (later...)

5. Implementing a Combinatorial Digital Logic Design

Drawing the digital logic design for combinatorial logic is very straight-forward.

Let's take the example of $F = A \cdot B + A \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot C$ which is presented in a sum-of-product form.

We'll need 1 **OR** gate that can take 3 inputs (it's the same idea as an **OR** gate with 2 inputs) and we'll need 3 **AND** gates (one with just 2 inputs, and the others with 3 inputs – again, it's the same idea).



6. The Multiplexer

The multiplexer or Mux, for short, is a *very commonly* used combinatorial logic device. It essentially functions as a “selector”. Take for example, its interpretation in C++ code:

```
bool mux(bool a, bool b, bool s) {
    if (s == 0)
        return a
    else return b
}
```

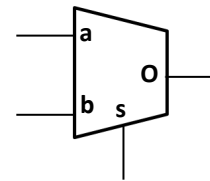
The truth table for a Mux is:

s	a	b	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

i.e.
input a

i.e.
input b

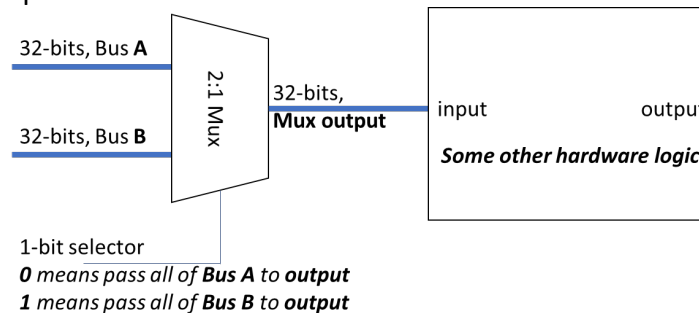
The symbol for a Mux (inputs **a**, **b**, **s**) is:



Inputs **a** and **b** are called “data inputs” and input **s** is called the “select” input. **O** is the “data output”.

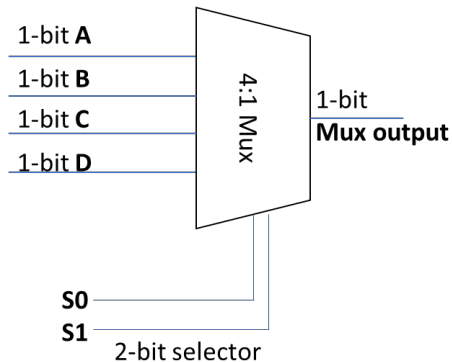
The simplest implementation of a Mux is what’s shown above, also known as a **1-bit 2:1 Mux**. This indicates that the mux takes in its data inputs **a** and **b** as *single bits* and then chooses which of the 2 of them will be presented to the output. Muxes ALWAYS have one output, however, Mux inputs and outputs can be made up of *multiple bits* as well.

For example, suppose that, inside a CPU, you have two (2) 32-bit buses (a “bus” is a collection of bits in hardware) that you want to be able to connect to some other hardware logic’s input:



The principle of that 32-bit, 2:1 mux is still the same as the simple 1-bit, 2:1 Mux. The only difference is that the data inputs and outputs of that Mux are “selected” over in 32-bit bunches at the same time. Note that the select (**s**) input is still just 1-bit because it’s used to select between input **A** (**s** = 0) and input **B** (**s** = 1), i.e. just 2 inputs.

Muxes can also be designed to choose between MORE than just 2 inputs. For example, a 1-bit, 4:1 Mux is one that picks between 4 inputs to be presented to the output.



Since we have to pick between 4 inputs, the number of bits for selection input has to be at least 2 bits, **S1** and **S0**. For example, if $S1 = 0$ and $S0 = 0$, then input **A** gets selected to go to the output; if $S1 = 0$ and $S0 = 1$, then input **B** gets selected to go to the output, and so on. The number of select bits needed is a function of how many inputs the mux has. It's essentially equal to $\log_2(\text{number of data inputs})$ rounded up to the next integer. For example, a 3-input mux needs 2 select bits (00 for **A**, 01 for **B**, 10 for **C**, and 11 doesn't get used).

The lecture will go over several other examples that use Muxes.

7. Karnaugh Maps

Karnaugh Maps (or **K-Maps**) are methods used to re-draw logic tables as to make the optimization of logic designs a lot easier than algebraic methods. Their limitation is that they are *ideal* for optimizing Boolean expressions of 2 to 4 input variables and they become more complicated for 5 or more input variables. In this class, we'll go over examples of K-Maps of 2 inputs and 3 inputs only. When implementing logic design optimization using modern software tools, these K-Map techniques (and others like them) are utilized by the software and are invisible to the typical designer (so we won't spend too much time on this concept).

The idea is to redraw the truth table in a 2-dimensional way, so for example, a 2-input truth table gets redrawn as shown:

A	B	f(A,B)
0	0	a
0	1	b
1	0	c
1	1	d

Classic Truth Table for 2 inputs

B \ A	0	1
0	a	c
1	b	d

K-Map for 2 inputs

Note how the inputs for $A = 0$ and $A = 1$ are shown on the horizontal, and the inputs for B on the vertical. This allows us to see all 4 combinations inside the cells.

A 2-input T.T. is trivial to convert to a K-Map. Let's see what a **3-input K-Map** looks like:

$\begin{array}{c} AB \\ C \end{array}$		$\overbrace{\hspace{1.5cm}}^A$			
		00	01	11	10
$\underbrace{\hspace{0.5cm}}^C$	0	0	2	6	4
	1	1	3	7	5
		$\underbrace{\hspace{1.5cm}}_B$			

K-Map for 3 inputs

Note: the horizontal shows *all 4 combinations* for **A** and **B**, **in this order**: **AB = 00, 01, 11, 10**. This ordering is **VERY IMPORTANT** (11 comes before the 10).

Every cell in this 8-cell K-Map (and it's 8-celled because there are 8 combinations of the 3 inputs) shows the relevant output. For example, the cell labeled "3" would show the output for the combination of **A = 0, B = 1, C = 1**.

8. Simplifying/Optimizing Logic Designs using Karnaugh Maps

Once you have a K-Map drawn out, you can begin the process of optimization.

Here's an example:

$$\text{Let } F = XZ + Z\bar{X} + ZXY$$

So, the Boolean expression for function **F** contains 3 inputs: **X, Y, and Z**. Therefore, we'll use the 3-input K-Map from the earlier section:

$\begin{array}{c} XY \\ Z \end{array}$		$\overbrace{\hspace{1.5cm}}^{X=0} \quad \overbrace{\hspace{1.5cm}}^{X=1}$			
		$\overbrace{\hspace{0.5cm}}^{Y=0}$	$\overbrace{\hspace{0.5cm}}^{Y=1}$	$\overbrace{\hspace{0.5cm}}^{Y=0}$	$\overbrace{\hspace{0.5cm}}^{Y=1}$
		00	01	11	10
	0				
	1				

Note that the choice to have variables **X** and **Y** be on the horizontal and variable **Z** to be on the vertical is completely arbitrary. Note also my annotation of the above (still empty) K-Map showing the columns of the horizontal that indicate where **X = 0** vs. **X = 1** and also where **Y = 0** vs. **Y = 1**.

The Boolean function **F** has 3 expressions:

- XZ**, is equal to **1** whenever **X=1** and also **Z=1**, so we'll update the K-Map thusly:

XY					
Z		00	01	11	10
0					
1				1	1

b) $Z\bar{X}$, is equal to 1 whenever $X=0$ and also $Z=1$, so we'll add to the K-Map thusly:

XY					
Z		00	01	11	10
0					
1		1	1	1	1

c) ZXY , is equal to 1 whenever $X=1$ and also $Y=1$, and also $Z=1$. Note that that confluence of the 3 variables, each equal to 1, is already represented, so the K-Map doesn't really change.

Now that the K-Map is completed, we can begin our analysis:

We note that we can "collect" all 4 horizontal 1s in the K-Map at the $Z=1$ row:

XY					
Z		00	01	11	10
0					
1		1	1	1	1

This is another way of saying that the expression in this K-Map is one of $F = Z$.

In other words, we've simplified the original expression of: $F = XZ + Z\bar{X} + ZXY$ to simply: $F = Z$

Could that be correct!!?? Let's verify this using algebraic methods:

$$\begin{aligned}
 F &= XZ + Z\bar{X} + ZXY \\
 &= Z(1 + \bar{X} + XY) = Z
 \end{aligned}$$

Before we get all excited about this new method, let's realize that this is just ONE example. Can we only "collect" 4 horizontal 1s? What are the rules in general!??

Optimizing with K-Maps has STRICT RULES:

1. Groups can contain **only 1s** (i.e. no 0s)
2. Only 1s in **adjacent** groups are allowed
3. Groups may **ONLY** be **horizontal or vertical** (no diagonals)
4. The number of 1s in a group must be a **power of two** (1, 2, 4, 8...)
5. Groups must be **as large AND as few** in numbers as “legally” possible
6. **Don’t leave any “1” behind!**
All 1s must belong to a group, even if it’s a group of one element
7. **Overlapping** groups are permitted
8. **Wrapping around** the map (left-right sides AND up-down sides) is permitted

Lecture will provide you with multiple examples to illustrate all these rules, which you are expected to know and utilize to simplify any 2 or 3 variable Boolean function/expression.